A Change Stream Ledger for MongoDB: Tamper Evidence Without a Ledger Engine

COMP.8157 Advanced Database Topics, Fall 2026: D.1 Project Preproposal

Olumide Ogunjebi
School of Computer Science
University of Windsor, Windsor, Canada
ogunjeb@uwindsor.ca

Rohith Nelluri
School of Computer Science
University of Windsor, Windsor, Canada
nellurir@uwindsor.ca

Yadu Pratap
School of Computer Science
University of Windsor, Windsor, Canada
Singh8a4@uwindsor.ca

Abstract. **Organisation that keep financial, medical, or regulatory records must be able to show that stored data has been silently altered, even by an administrator. Relational engines have begun of offer this natively, as SQL Server does with its Ledger feature, but MongoDB provides no cryptographic tamper evidence, and Amazon QLDB, a managed ledger built around documents was retired in 2025. This project asks whether a stock MongoDB deployment can be given ledger guarantees through its public interfaces alone. We will build two designs, one that hashes each change inside the writing transaction and one that builds a merkle chained ledger asynchronously from MongoDB change streams, and compare their cost exposure window, and attack detection with SQL Server Ledger on the same workloads.**
Index Terms. Tamper evidence, ledger databases, document databases, MongoDB Server Ledger, database forensics.

1.CORE IDEA

The problem we study is how to give a general purpose document database the tamper evidence that ledger databases provide, without changing its engine, routing writes through a third party service, or relying on trusted hardware. MongoDB already exposes two mechanisms we can build: multi document transactions, which let an application write a document and its hash atomically, and change streams, which deliver every majority committed change recorded in the replica set oplog. On each mechanism we will build a ledger that hashes every document change, groups transactions into Merkle blocks, chains the blocks and periodically publishes a short digest to storage that administrator cannot rewrite, while a verifier checks the chain and compares every current document with the state the ledger implies. To judge how close this retrofit comes to a ledger built into the engine, we will run the same workloads on SQL Server Ledger as a reference. Finally, we will subject all configurations to a set of insider attacks and record which ones each detects and what evidence it leaves.

2.NOVELTY CLAIM

The assumption we challenge is that cryptographic tamper evidence requires a purpose built ledger system [1], [2], changes inside the database engine [3], [4], [5], or trusted hardware [6]. That assumption leaves document databases with no practical option: MongoDB has no native ledger, the commercial ProvenDB service placed its own layer in the write path and anchored proofs on a public blockchain [7], and forensic work on MongoDB detects tampering only after the fact by comparing database states [8]. The most systematic benchmark of verifiable databases, VeriBench [9], compared continuous and deferred verification across six systems, but every system it measured, including SQL Ledger, was purpose built or reimplemented for the study, so it cannot tell a MongoDB user what verifiability would cost on the engine they already run.

Our claim is the public interfaces of MongoDB are enough to provide ledger guarantees for committed data comparable to those of SQL Server, and that the choice between synchronous and asynchronous capture is a real trade off that has not been measured on a commodity document store. Hashing inside the transaction keeps the ledger current, but it adds work to every write, makes concurrent transactions contend for the head of the chain, and leaves only an unexplained mismatch when a client writes around the ledger code. Change streams move hashing off the write path and see every change recorded in the oplog whichever client made it, which preserves forensic evidence, but the ledger then trails the database by a window in which a change is committed but not yet chained, and changes that never reach the oplog escape it. Quantifying that window against the throughput it buys, placing both designs beside a native ledger, and mapping each configuration to the attacks it detects is the new insight this project aims to provide.

1. RELATED WORK

3.1Closest Related Work

The closest work is SQL Ledger by Antonopoulos et al. [3], now part of SQL Server and Azure SQL Database. Rows changed by a transaction are hashed, transaction hashes are combined into Merkle trees that form chained blocks, earlier row versions are kept in history tables, and database digests are stored outside the database so that a verifier can recompute the hashes and combined into Merkle trees that form chained blocks, earlier row versions are kept in history tables and database digests are stored outside the database so that a verifier can recompute the hashes and detect tampering. Our ledger follows this structure, which is why SQL Ledger is built into a relational engine, whereas we bring comparable guarantees to a document database through its public interfaces, without engine changes. Second, SQL Ledger offers one engine integrated design, whereas we build a synchronous and an asynchronous variant and measure the trade off between them against SQL Ledger itself. Third, we evaluate every configuration against a concrete set of insider attacks and report which ones each detects and what evidence it leaves.

TABLE I
WHERE PRIOR WORK PLACES VERIFIABILITY

Work	Mechanism	Engine unchanged	Trusted element
LedgerDB [1]	Purpose built ledger	No	Time authority
SQL Ledger [3]	Inside SQL Server	No	Digest storage
Nathan [4]	Modified PostgreSQL	No	Peer nodes
VeriDB [6]	SGX enclave	No	Intel SGX
GlassDB [2]	Purpose built ledger	No	Auditors
VeriBench [9]	Benchmark	Rebuilt	Varies
ProvenDB [7]	MongoDB service	Yes	Bitcoin anchor
Chopade [8]	State comparison	Yes	None
This project	**Change streams**	**Yes**	**Digests**

B. Comparative Positioning

Purpose built systems obtain strong guarantees by owning the whole stack. LedgerDB [1] is a centralized ledger database from Alibaba that anchors accumulated Merkle trees and GlassDB [2] is a distributed ledger database that authenticates database states rather than transactions to obtain compact proofs, with auditors checking the service. Amazon QLDB, a managed ledger for document data, reached end of support in July 2025 [10]. Each of them requires migrating to a new engine.

A second line of work changes an existing engine. Snodgrass et al. [11] hashed transactions and sent digests to an external notary to expose tampering with audit logs, through an extension built into MySQL with Berkeley DB. Nathan et al. [4] added about 4,000 lines of C to PostgreSQL to build a blockchain relational database for mutually distrustful organizations, and Xia et al. [5] combine authenticated data structures with a deterministic, batch based concurrency control scheme so that clients can verify the ACID properties of an outsourced database. VeriDB [6] instead relies on Intel SGX hardware. Each of these results depends on engine code or hardware that typical MongoDB user cannot deploy.

VeriBench [9] is the closest evaluation study. It shows that deferred verification with batching can raise throughput well above continuous verification, which motivates our asynchronous design, yet it measures only purpose built or reimplemented systems and focuses on performance rather than on tracks. Work on MongoDB Atlas that hashes documents into Merkle trees and anchors the roots on the Bitcoin blockchain, but it is a commercial layer in the write path, and we found no published evaluation of its overhead. Chopade and Pachghare [8] detect tampering in MongoDB by comparing old and new database states, which supports investigation after an incident but gives no cryptographic proof for individual changes. Our design differs from all of these by building a verifiable chain of every committed change from the database's own replication stream, with nothing added to the write path Table I summarizes where each approach places verifiability.

2. PLANNED APPROACH AND FEASIBILITY

Everything will run on one laptop with free software: MongoDB 8.0 Community Server as a single node replica set, SQL Server 2022 in its free Developer or Express edition, Python with the pymongo and pyodbc drivers, and Docker for repeatable setups. The synchronous design will wrap each application write in a multi document transaction that also appends a ledger entry holding the SHA256 hash of the canonical document and of the previous entry. The asynchronous design will turn on the collection option that records the before and after image of each document, and a small Python service will read the change stream, group events by transaction, compute a Merkle root per transaction, chain the roots into blocks, and write a digest every few seconds to an append only file outside the database's control. The verifier will check the chain against the digests, detect breaks in the stream, and compare every current document with the ledger. On SQL Server, the same data will be held in updatable ledger tables, with digests generated and verified through the built in ledger procedures.

Performance will be measured with YCSB, which has bindings for MongoDB and for JDBC databases such as SQL Server, and with a small banking workload modelled on SmallBank that we will implement for both systems, as VeriBench did [9]. Each configuration will be compared with an unprotected baseline at several client counts in throughput, tail latency, storage growth, and the lag

between commit and digest. Security will be evaluated with a scripted attack suite run by an administrator: direct writes that bypass the ledger code, edits to the ledger collection, restarting the server outside replica set mode and writing to it, letting the oplog roll over while the ledger service is stopped, and restoring an older backup. For each attack we will record whether it is detected, how quickly, and what evidence remains. One member will lead the synchronous design and SQL Server baseline, one the change stream ledger, and one the verifier and attacks, with milestones through the final demonstration on 29 November.

5.EXPECTED OUTCOMES AND IMPACT

We expect three outcomes. The first is an open source ledger layer for MongoDB, in two variants, that an organization could deploy on an existing replica set without changing the database. The second is a measured comparison of synchronous capture, asynchronous capture, and a native engine ledger, expressed as throughput cost against exposure window, which extends the comparison of VeriBench to a real document store and a real production ledger. The third is an attack matrix showing which forms of insider tampering each configuration detects and what forensic evidence it preserves, which is useful to auditors and investigators. If the change stream design proves nearly free while matching SQL Server Ledger in detection, ledger functionality could live in the replication stream of an existing database.

REFERENCES

[1] X. Yang, Y. Zhang, S. Wang, B. Yu, F. Li, Y. Li, and W. Yan, "LedgerDB: A centralized ledger database for universal audit and verification," *Proc. VLDB Endow.*, vol. 13, no. 12, 2020, doi: 10.14778/3415478.3415540.

[2] C. Yue, T. T. A. Dinh, Z. Xie, M. Zhang, G. Chen, B. C. Ooi, and X. Xiao, "GlassDB: An efficient verifiable ledger database system through transparency," *Proc. VLDB Endow.*, vol. 16, no. 6, 2023, doi: 10.14778/3583140.3583152.

[3] P. Antonopoulos, R. Kaushik, H. Kodavalla, S. Rosales Aceves, R. Wong, J. Anderson, and J. Szymaszek, "SQL Ledger: Cryptographically verifiable data in Azure SQL Database," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2021, doi: 10.1145/3448016.3457558.

[4] S. Nathan, C. Govindarajan, A. Saraf, M. Sethi, and P. Jayachandran, "Blockchain meets database: Design and implementation of a blockchain relational database," *Proc. VLDB Endow.*, vol. 12, no. 11, 2019, doi: 10.14778/3342263.3342632.

[5] Y. Xia, X. Yu, M. Butrovich, A. Pavlo, and S. Devadas, "Litmus: Towards a practical database management system with verifiable ACID properties and transaction correctness," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2022, doi: 10.1145/3514221.3517851.

[6] W. Zhou, Y. Cai, Y. Peng, S. Wang, K. Ma, and F. Li, "VeriDB: An SGX-based verifiable database," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2021, doi: 10.1145/3448016.3457308.

[7] J. Vaughan, "ProvenDB brings blockchain applications to world of MongoDB," *TechTarget SearchDataManagement*, May 2019. [Online]. Available: https://www.techtarget.com/data-technologies/news/252462934/ProvenDB-brings-blockchain-applications-to-world-of-MongoDB

[8] R. Chopade and V. Pachghare, "Performance analysis of proposed database tamper detection technique for MongoDB," in *Proc. 5th Int. Congr. Information and Communication Technology (ICICT)*, Springer, 2021, doi: 10.1007/978-981-15-5856-6_39.

[9] C. Yue, M. Zhang, C. Zhu, G. Chen, D. Loghin, and B. C. Ooi, "VeriBench: Analyzing the performance of database systems with verifiability," *Proc. VLDB Endow.*, vol. 16, no. 9, 2023, doi: 10.14778/3598581.3598588.

[10] R. Losio, "AWS discontinues Amazon Quantum Ledger Database (QLDB)," *InfoQ*, Jul. 2024. [Online]. Available: https://www.infoq.com/news/2024/07/aws-kill-qldb

[11] R. T. Snodgrass, S. S. Yao, and C. Collberg, "Tamper detection in audit logs," in *Proc. 30th Int. Conf. Very Large Data Bases (VLDB)*, 2004.